SOFTWARE ENGINEERING · SOMMERVILLE, 10TH EDITION — CHAPTER 2

2.1 Process Models & 2.2 Process Activities

The complete, exhaustive explanation — every definition, stage, diagram and side-box — so you never need to open the textbook again.

Includes fully solved Exercises 2.1 · 2.3 · 2.4 · 2.6 · 2.8

Deep-dive revision deck

2.1 — DEFINITION

What Exactly Is a Software Process?

A software process is **a set of related activities that leads to the production of a software system**. Because there are many different types of software systems, there is **no universal software engineering method** applicable to all of them — and therefore **no universally applicable process**. The process used in different companies depends on: the **type of software** being developed, the **requirements of the customer**, and the **skills of the people** writing the software.

Every process must include these 4 fundamental activities, in some form:

1. Specification
The functionality of the software and constraints on its operation must be defined.

2. Development
The software to meet the specification must be produced.

3. Validation
The software must be validated to ensure it does what the customer wants.

4. Evolution
The software must evolve to meet changing customer needs.

These are **complex activities in themselves**, with sub-activities such as requirements validation, architectural design, and unit testing. Processes also include supporting activities like **configuration management** and **project planning**.

Safety-critical systems → very structured process · Rapidly changing business systems → flexible, agile process

Describing a Process: 3 Extra Dimensions

When we describe processes, we usually talk about the **activities** and their **ordering**. But a full description also needs to say **who is involved**, **what is produced**, and **what conditions** influence the sequence:

1. Products / Deliverables

The **outcomes** of a process activity. Example: the outcome of architectural design may be a model of the software architecture.

2. Roles

Reflect the **responsibilities** of the people involved. Examples: project manager, configuration manager, programmer.

3. Pre- & Postconditions

Conditions that must hold **before** and **after** an activity. Example precondition: all requirements are approved before architectural design begins. Example postcondition: the UML models describing the architecture have been reviewed.

Software processes are **complex** and, like all intellectual/creative processes, rely on **people making decisions and judgments**. Because no single process is right for every kind of software, most companies have developed their **own variant** processes.

Plan-driven processes: all activities are planned in advance, progress is measured against this plan. **Agile processes** (Ch.3): planning is incremental and easier to change. For **large systems**, you typically need a **balance** between plan-driven and agile.

2.1 CONTINUED

What Is a "Process Model"?

A software process model (sometimes called a **Software Development Life Cycle / SDLC model**) is a **simplified representation** of a software process, presented from a particular perspective. Each process model only provides **partial information** — e.g., a process activity model shows the activities and their sequence but may not show the roles of people involved.

The three general models covered here (sometimes called **process paradigms**) are presented from an **architectural perspective** — i.e., we see the **framework** of the process, not the fine-grained detail of its activities. Think of them as **process frameworks** that get extended and adapted into specific, real engineering processes.

1 · Waterfall

2 · Incremental Development

3 · Integration & Configuration

2.1.1 — THE WATERFALL MODEL, IN FULL

The 5 Stages, Explained One by One

Derived from engineering process models used in **large military systems engineering** (Royce, 1970) — the first published software process model. Named "waterfall" because of the cascade from one phase to the next. It is a **plan-driven process**: in principle, you plan and schedule **all** activities before development starts.

1. Requirements analysis & definition — the system's services, constraints and goals are established by consultation with system users, then defined in detail to serve as a system specification.

2. System and software design — the systems design process allocates requirements to hardware or software systems, establishing an overall system architecture. Software design identifies and describes the fundamental software abstractions and their relationships.

3. Implementation and unit testing — the software design is realized as a set of programs or program units; unit testing verifies that each unit meets its specification.

4. Integration and system testing — individual program units/programs are integrated and tested as a complete system to ensure the software requirements have been met; the system is then delivered to the customer.

5. Operation and maintenance — normally the **longest** life-cycle phase. The system is installed and put into practical use. Maintenance involves correcting undiscovered errors, improving unit implementations, and enhancing services as new requirements are discovered.

Why the Waterfall Breaks Down in Practice

The theory

In principle, each phase's result is one or more documents that must be **approved ("signed off")** before the next phase starts. The following phase should not begin until the previous one has finished — this works well for hardware, where manufacturing costs are high.

The reality

For **software**, stages overlap and feed information back to each other. During design, problems with requirements are identified; during coding, design problems are found. The process is **never a simple linear model** — it involves feedback from one phase to another.

The danger: premature freezing of requirements

When new information emerges, earlier documents should be modified to reflect the required changes — but this needs **customer approval** and delays the project. As a result, both customers and developers may **freeze the specification early** so no further changes are made. This means problems are **left for later resolution, ignored, or "programmed around" —** which may mean the system doesn't do what the user wants, and may lead to badly structured systems as design problems are circumvented with implementation tricks.

During the final phase (operation & maintenance), errors and omissions in the original requirements are discovered, program/design errors emerge, and new functionality is needed — so the system must **evolve to remain useful**, which may involve repeating earlier process stages.

Key insight: software's flexibility is exactly what the waterfall model struggles to accommodate

When Waterfall Is (and Isn't) Appropriate

1. Embedded systems

The software interfaces with hardware. Because hardware is inflexible, it's usually impossible to delay decisions on the software's functionality until implementation.

2. Critical systems

Need extensive safety/security analysis of specification & design — these documents must be complete so the analysis is possible. Problems found at implementation are very expensive to fix.

3. Large multi-company systems

Hardware in such systems may follow a similar model; complete specifications are needed so different partner companies can independently develop subsystems.

It is **not** the right model where informal team communication is possible and requirements change quickly — iterative development and agile methods fit better there.

Variant: Formal System Development

A mathematical model of the specification is created, then **refined through mathematical transformations** that preserve consistency, into executable code (e.g., the **B method**, Abrial 2005/2010). Mostly used for software with **stringent safety, reliability, or security requirements** — it simplifies producing a safety/security case proving the system meets its requirements. Rarely used elsewhere due to the **high cost** of developing a formal specification.

Directly answers Exercise 2.1 — see the worked solution later in this deck

A "UNIVERSAL" MODEL

The Rational Unified Process (RUP)

Developed by Rational, a U.S. software engineering company (Krutchen, 2003). The RUP **brings together elements of ALL general process models** and supports both prototyping and incremental delivery of software. It can be **instantiated in different ways** to create processes resembling any of the general models. It has been adopted by some large companies (notably **IBM**), but has not gained widespread acceptance.

Described from 3 perspectives

Dynamic perspective
Shows the **phases** of the model over time.

Static perspective
Shows the process **activities** enacted.

Practice perspective
Suggests **good practices** to use.

4 phases of the dynamic/time view

Inception
Business case established.

Elaboration
Requirements & architecture developed.

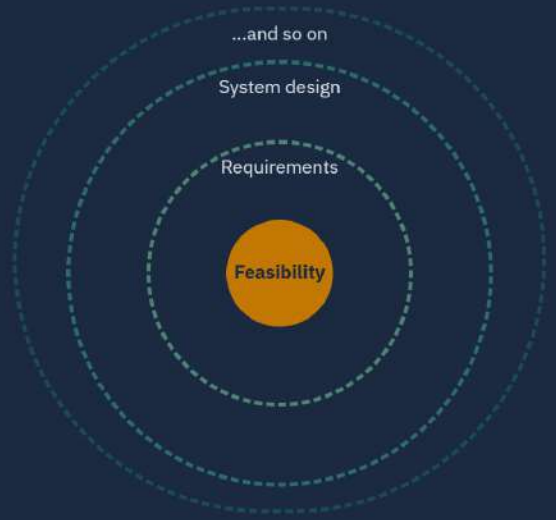
Construction
The software is implemented.

Transition
System is deployed into use.

For large systems engineering, it makes sense to combine the best features of all general models

A RISK-DRIVEN VARIANT

Boehm's Spiral Process Model



Proposed by **Barry Boehm** (1988), one of software engineering's pioneers. It is an **incremental process model** that is **risk-driven** rather than document-driven.

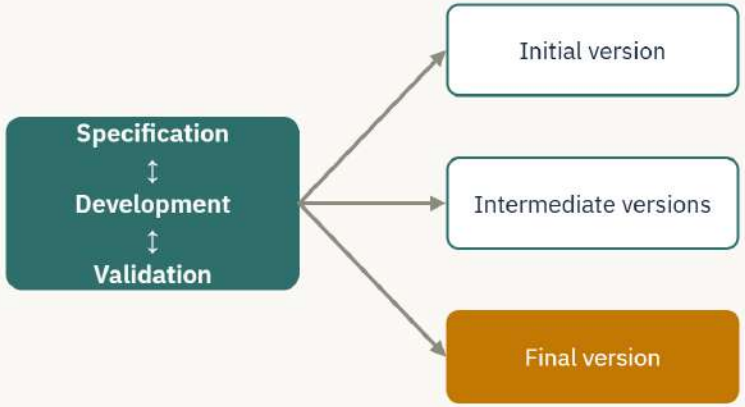
The process is represented as a **spiral rather than a sequence of activities**. Each loop in the spiral represents **one phase** of the software process: the innermost loop might concern system **feasibility**, the next loop **requirements definition**, the next loop **system design**, and so on.

The spiral model **combines change avoidance with change tolerance**, and it includes **explicit risk-management activities** specifically to reduce the risks that changes to the project create.

2.1.2 — INTERLEAVED

Incremental Development, Fully Explained

Based on the idea of developing an **initial implementation**, getting **feedback** from users and others, and **evolving the software through several versions** until the required system has been developed. Specification, development and validation activities are **interleaved rather than separate**, with rapid feedback across activities.



Generally, **early increments include the most important/urgently needed functionality**. This means the customer can evaluate the system at a relatively early stage to check it delivers what's required — if not, only the current increment needs changing, with new functionality possibly defined for later increments.

Now the **most common approach** for application systems and products — can be **plan-driven, agile, or a mixture**. In plan-driven, increments are identified in advance; in agile, only the early increments are identified, and later ones depend on progress and customer priorities.

Reflects how we actually solve problems — a series of steps, backtracking when we spot a mistake

3 Advantages Over the Waterfall Model

1. The cost of implementing requirements changes is reduced
The amount of analysis and documentation that has to be redone is significantly less than with the waterfall model, because only the current (and future) increments are affected.

2. It is easier to get customer feedback on the work done
Customers can comment on demonstrations of the software and see how much has been implemented — they find it difficult to judge progress from design documents alone.

3. Early delivery & deployment of useful software is possible
Even if all functionality hasn't been included, customers can use the software and gain value earlier than a waterfall process allows.

Note: incremental development **does not require** delivering each increment to the customer — you may develop increments and expose them to stakeholders for comment without deploying into their environment. ("Incremental delivery", §2.3.2, means the software is used in real operational processes.)

Management Problems with Incremental Development

1. The process is not visible

Managers need regular deliverables to measure progress. If systems are developed quickly, it is **not cost-effective** to produce documents that reflect every version.

2. System structure tends to degrade

Regular change leads to **messy code** as new functionality is added however possible. It becomes increasingly difficult/costly to add features. Agile fix: regularly **refactor** (improve & restructure) the software.

Bureaucracy conflicts (a real-world complication)

Large organizations have **bureaucratic procedures** that evolved over time, which may mismatch a more informal, iterative/agile process. Sometimes these procedures exist for good reason — e.g., ensuring compliance with external regulations such as the U.S. **Sarbanes–Oxley accounting regulations**. Changing such procedures may not be possible, so **process conflicts can be unavoidable**.

Scaling to large, complex, long-lifetime systems

These problems become particularly acute here, since different teams develop different parts of the system. Large systems need a **stable framework/architecture**, with each team's responsibilities clearly defined against that architecture — this has to be **planned in advance**, not developed incrementally.

Directly relevant to Exercise 2.2's discussion of unclear requirements

2.1.3 — BUILD FROM PARTS

Integration & Configuration — All 5 Stages

In most projects there is **informal reuse** — developers search for or know of similar code, modify it, and integrate it. Since 2000, **formal reuse-oriented processes** have become widely used, relying on a base of reusable components and an integrating framework.

1. **Requirements specification** — initial requirements are proposed; they don't need elaborate detail, just brief descriptions of essential requirements and desirable features.

2. **Software discovery & evaluation** — given the outline requirements, a search is made for components/systems providing the needed functionality; candidates are evaluated for essential-requirement fit and general suitability.

3. **Requirements refinement** — requirements are refined using information about the discovered components; the specification is re-defined to reflect what's actually available. Where impossible, the discovery/evaluation stage may be re-entered to search for alternatives.

4. **Application system configuration** — if an off-the-shelf system meeting the requirements is available, it is configured for use to create the new system.

5. **Component adaptation & integration** — if there's no off-the-shelf system, individual reusable components may be modified, new components developed, and everything integrated to create the system.

Reuse: Trade-offs & Component Types

Advantages

Reduces the amount of software to be developed, so lowers **cost and risk**, and usually leads to **faster delivery**.

Disadvantages

Requirements compromises are inevitable — the system may not meet real user needs. Some control over system evolution is lost, since reusable components evolve outside the organization's control.

Three types of reusable component

1 · Stand-alone systems
General-purpose, feature-rich; adapted/configured for one application.

2 · Object collections
Developed as a component/package for a framework, e.g., Java Spring.

3 · Web services
Built to service standards; available for remote invocation over the Internet.

Further reading: reuse (Ch.15), component-based SE (Ch.16–17), service-oriented systems (Ch.18)

2.2 — PROCESS ACTIVITIES

Real Processes: Interleaved, Tool-Supported Activity

Real software processes are **interleaved sequences of technical, collaborative and managerial activities** with the overall goal of specifying, designing, implementing and testing a software system. Processes are now generally **tool-supported** — developers use requirements management systems, design model editors, program editors, automated testing tools and debuggers.

The four basic activities — specification, development, validation, evolution — are organized **differently in different processes**: in sequence for waterfall, interleaved for incremental development. How they're carried out depends on the **type of software**, the **experience/competence of the developers**, and the **type of organization**.

01
Specification

02
Design & Implementation

03
Validation

04
Evolution

2.2.1 — SPECIFICATION

Software Specification = Requirements Engineering

The process of **understanding and defining** what services are required from the system and the constraints on its operation/development. A **particularly critical stage** — mistakes made here inevitably lead to later problems in design and implementation. A company may first run a cheap, short-term **feasibility study** to check whether there's a need/market and whether the software is technically & financially realistic, before detailed requirements work begins.

Requirements are usually presented at **two levels of detail**: end-users/customers need a **high-level statement**; system developers need a **more detailed system specification**.

1 · Requirements elicitation & analysis — deriving requirements through observation of existing systems, discussions with users/procurers, task analysis, etc. May involve developing models/prototypes to help understand the system to be specified.

2 · Requirements specification — translating gathered information into a document defining a set of requirements. Two types: **user requirements** (abstract statements for customer/end-user) and **system requirements** (a more detailed description of functionality).

3 · Requirements validation — checking requirements for realism, consistency, and completeness. Errors are inevitably discovered during this process and the document must then be modified to correct them.

Analysis, definition and specification are **interleaved**, and requirements analysis continues throughout. In **agile methods**, specification is **not a separate activity** — requirements are informally specified per increment, just before it is developed, according to user priorities, with elicitation coming from users on/close to the development team.

2.2.2 — DEVELOPMENT

Software Design & Implementation — the Full Model

The implementation stage is the process of developing an **executable system** for delivery to the customer. In agile approaches, design and implementation are interleaved with **no formal design documents** — the software is still designed, but informally, on whiteboards and in notebooks. A software design describes the **structure** to be implemented, the **data models/structures** used, the **interfaces** between components and, sometimes, the **algorithms**. Designers add detail in **stages**, with constant backtracking to revise earlier decisions — **design rework is inevitable** as new information is generated.

Architectural design
Identifies the overall structure of the system, principal components (subsystems/modules), their relationships, and how they're distributed.

Database design
Designs the system data structures and how they're represented in a database — depends on whether an existing database is reused or a new one created.

Interface design
Defines the interfaces between components. Must be unambiguous so a component can be used by others without knowing how it's implemented.

Component selection & design
Search for reusable components; if none suitable, design new ones — from a simple description to a detailed UML model that can auto-generate an implementation.

Inputs to design: platform information, software requirements, data descriptions. **Outputs**: system architecture, database design, interface specification, component descriptions — for critical systems these are detailed documents; with agile, they may just be the code itself. Activities are **interleaved and interdependent** — e.g. an architectural choice constrains database & interface design.

From Design to Code: Programming & Debugging

Program development follows naturally from system design. Though some classes of program (e.g., safety-critical systems) are usually designed in detail before implementation, it's more common for design and programming to be **interleaved**. Tools may generate a **skeleton program** from a design, including code to define/implement interfaces — the developer then often just adds the operation details of each component.

Programming is individual — there's no single "right" process

Some programmers start with components they **understand** and develop these first, moving to less-understood ones later. Others do the **opposite** — leaving familiar components till last.

Some developers **define data early** and use it to drive development; others leave data unspecified for as long as possible.

Testing
Establishes the **existence** of defects (bugs) in a program.

Debugging
Is concerned with **locating and correcting** those defects — a different process from testing.

The debugging process: generate **hypotheses** about the observable (anomalous) program behavior → test those hypotheses to find the fault that caused the anomaly. May involve manually **tracing the code**, and may need **new test cases** to localize the problem. **Interactive debugging tools**, which show intermediate variable values and a trace of executed statements, are normally used to support this process.

2.2.3 — VALIDATION

Software Validation: the 3-Stage Testing Process

Validation (V&V — **verification & validation**) shows a system both **conforms to its specification** and **meets the customer's expectations**. Program testing (executing the system with simulated data) is the principal technique; validation may also involve inspections and reviews at every stage. Most V&V effort, however, is spent on program testing. Except for small programs, systems should **not** be tested as one monolithic unit.

1 · Component testing
Components are tested independently by the people developing them, without other system components. May be simple entities (functions) or coherent groupings (object classes). Automated tools like **JUnit** rerun tests whenever a new component version is created.

2 · System testing
Components are integrated into a complete system. Concerned with finding errors from **unanticipated interactions** and interface problems, and with showing the system meets its functional and **non-functional** requirements, including emergent properties. For large systems, a multistage process: components → subsystems → final system.

3 · Customer testing
The final stage before acceptance — tested by the customer with **real data**, not simulated test data. May reveal requirements errors/omissions, or that facilities don't meet real needs, or unacceptable performance. For products, this reveals how well it meets customer needs (**"beta testing"**).

Ideally, component defects are found early and interface problems when integrated — but as defects are discovered, the program must be **debugged**, possibly repeating earlier test stages. It's an **iterative process** with feedback from later stages to earlier ones.

Two Testing Philosophies: V-Model vs. TDD

Plan-driven: the V-model

When a plan-driven process is used (e.g., critical systems), testing is driven by a set of **test plans**, developed by an independent team from the system specification and design. Test plans link testing and development: **customer test plan** ↔ requirements specification, **system integration test plan** ↔ system specification, **sub-system integration test plan** ↔ system design. This is sometimes called the **V-model** of development — turn the diagram on its side to see the "V" shape, showing which validation activity corresponds to which development stage.

Agile: test-driven development (TDD)

A normal part of agile processes: **tests are developed before the code**, along with the requirements, for each increment. This helps testers and developers **understand the requirements** properly and ensures there are **no delays** waiting for test cases to be created once code exists. If an incremental approach is used generally, each increment should be tested as it's developed, with tests based on that increment's requirements.

Normally, **component testing is simply part of the normal development process**: programmers make up their own test data and incrementally test code as it is developed — the programmer knows the component best and is therefore the **best person to generate its test cases**. But system and customer testing need an **independent perspective**.

TESTING A MARKETED SOFTWARE PRODUCT

Beta Testing

When a system is to be **marketed as a software product** (rather than custom-built for one customer), a testing process called **beta testing** is often used.

1. Delivered to a number of **potential customers** who agree to use it.

2. They **report problems** to the developers as they use it.

3. Exposes the product to **real use**, detecting errors developers hadn't anticipated.

4. The product may be **modified** and released for further beta testing or for general sale.

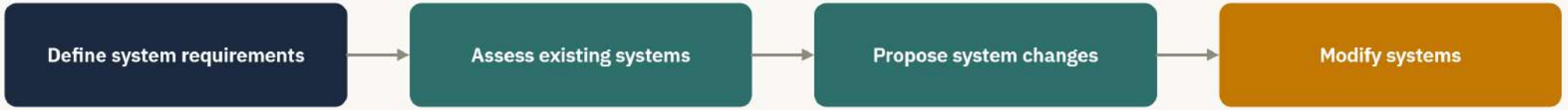
Contrast with custom-software customer testing: there the system is tested with the one customer's real data, which may reveal requirements errors/omissions that simulated test data never exercised.

2.2.4 — EVOLUTION

Software Evolution

The **flexibility of software** is one of the main reasons more and more software is incorporated into large, complex systems. Once a decision is made to manufacture hardware, it's very expensive to change the hardware design — but software can change **at any time** during or after development, and even extensive changes are much cheaper than corresponding hardware changes.

Historically there's always been a split between the process of **software development** (a creative activity, from initial concept to working system) and **software evolution/maintenance** (sometimes seen as dull, uninteresting, and less challenging). This distinction is **increasingly irrelevant** — very few systems are completely new, so it's more realistic to see software engineering as a **continuum, an evolutionary process**.



This loop repeats as software is **continually changed** over its lifetime, in response to changing requirements and new customer needs — "Modify systems" feeds a "New system" which again becomes an "Existing system" to be assessed.

SUMMARY

Everything from 2.1 & 2.2, Condensed

A software process = activities producing a software system; a process **model** is an abstract, partial representation of it.

Waterfall: 5 sequential, signed-off phases — best for embedded, critical, multi-partner systems.

Incremental development: specify/develop/validate interleaved, built as versions — now the most common approach.

Integration & configuration: 5-stage reuse-driven model built around discovering & adapting existing components.

RUP & **Spiral model** combine elements of all three, adding phases/risk-driven loops.

Requirements engineering: elicitation & analysis → specification → validation; user vs. system requirements.

Design & implementation: architecture, database, interface, component design — interleaved & interdependent.

Testing: component → system → customer (V-model or TDD); testing ≠ debugging; beta testing for products.

Evolution: software's flexibility lets it change cheaply at any time — development and maintenance are now seen as one continuous, evolutionary process.

FULLY SOLVED

Exercises 2.1 · 2.3 · 2.4 · 2.6 · 2.8

Complete, exam-ready answers — not just hints — for the exercises you asked for, each one grounded directly in the 2.1 and 2.2 material above.

2.1 Process model choice | 2.3 Reuse & requirements | 2.4 User vs. system reqs | 2.6 Testing & who tests | 2.8 Prototype report

EXERCISE 2.1 — SOLVED

Choosing the Right Process Model for 4 Systems

a) Antilock braking control system → Waterfall model (possibly with the formal-methods/B-method variant). It's an **embedded, safety-critical, real-time** system: it must interface tightly with hardware (brakes, sensors) so functionality can't be left open until implementation, and it needs extensive safety analysis of a complete specification & design before coding — exactly the two scenarios where waterfall fits.

b) A VR system to support software maintenance → Incremental development (with prototyping). This is a **novel, exploratory** application domain — nobody yet fully knows what a good VR maintenance tool looks like, so requirements will only become clear as users try early versions and give feedback. Freezing a full specification up front (waterfall) would be premature and risky.

c) A university accounting system replacing an existing system → Reuse-oriented (integration & configuration). Accounting is a **well-understood, standardized** domain with many mature off-the-shelf packages; replacing an existing system also means the requirements are largely already known from how the current system behaves — so configuring a proven package is far more efficient than building from scratch.

d) An interactive travel-planning system with lowest environmental impact → Incremental / agile development. This is a **novel, user-facing product** with evolving requirements — the exact UX and the "lowest-impact" scoring algorithm will need iteration based on real user feedback, which incremental delivery supports much better than an up-front, fixed specification.

EXERCISE 2.3 — SOLVED

Why Repeat Requirements Engineering in the Reuse Model?

Question: Consider the integration & configuration process model (Fig. 2.3). Explain why it's essential to repeat the requirements engineering activity in the process.

Answer: The candidate components or systems discovered during "software discovery and evaluation" will **almost never match the original requirements exactly**. They are general-purpose or off-the-shelf, built for someone else's needs, so there is inevitably a gap between what was originally specified and what is actually available.

Requirements engineering must therefore be **re-entered at the "requirements refinement" stage**: the requirements document is modified to reflect what the discovered components can realistically provide — accepting reasonable compromises rather than insisting on the original wish-list.

If, after refinement, **no acceptable compromise is possible** (the gap is too large), the process loops **back to "software discovery and evaluation"** to search for alternative components — which again forces another round of requirements refinement once new candidates are found.

In short: unlike the waterfall model, where requirements are fixed once at the start, in the reuse model requirements engineering is an **iterative, interleaved activity** that keeps adjusting itself to match the reality of what reusable software is actually on offer.

EXERCISE 2.4 — SOLVED

Why Separate User Requirements from System Requirements?

Question: Suggest why it is important to make a distinction between developing user requirements and developing system requirements.

1. Different audiences, different language. User requirements are abstract statements in natural language/diagrams that **customers, managers and non-technical end-users** can read and validate. System requirements are precise, detailed technical specifications that **developers** need to design and build the system unambiguously. Merging the two produces a document that's either too technical for the customer to confirm, or too vague for engineers to implement correctly.

2. Contracts and agreement. User requirements typically form the basis of the **contract/agreement** with the customer — what they're actually paying for and signing off on. If this were mixed with deep technical detail, the customer could unknowingly agree to things they don't understand, or disputes could arise over implementation-level language they never meant to constrain.

3. Traceability and change management. Keeping two levels lets each detailed system requirement be **traced back** to the user need it serves. This makes it far easier to validate that nothing has been over-engineered (a system requirement with no matching user need) and nothing has been missed (a user need with no supporting system requirement) — and it makes impact analysis of a change request much more manageable.

EXERCISE 2.6 — SOLVED

Why Staged Testing? Are Programmers the Best Testers?

Question: Explain why software testing should always be an incremental, staged activity. Are programmers the best people to test the programs they have developed?

Part 1 — Why staged/incremental testing?

Testing a whole system as one monolithic unit makes it extremely hard to **isolate where a failure originates**, since many components interact. Staged testing — **component → system → customer** — catches different defect types at the **earliest, cheapest** point: component testing finds logic errors inside a unit; system testing finds interface mismatches and emergent behavior from component interaction; customer testing finds real requirements gaps that only genuine usage/data reveal. A bug caught at component stage is far cheaper to fix than one found after full integration or, worse, after delivery, because more code has since been built on top of it.

Part 2 — Are programmers the best testers of their own code?

For **component testing**, yes — the programmer knows the code's internal logic intimately and can write good unit test cases quickly. But for **later stages, generally no**, because: (a) they share the **same misunderstandings** that caused the bug, so their tests may just re-confirm the wrong assumption; (b) there's a natural **psychological reluctance** to rigorously try to "break" one's own work; (c) an **independent tester** brings a fresh, objective perspective and is more likely to try edge cases and real user scenarios the author never considered. In practice: programmers do component testing of their own code, while an independent team (or the customer) handles system/customer testing.

EXERCISE 2.8 — SOLVED (1/2)

Short Report: Why the Prototype Shouldn't Become the Production System

Scenario: You've built a prototype; your manager is impressed and wants to put it into production, adding features as needed, to save the cost of "proper" development. Write a short report explaining why this is a bad idea.

To: Project Manager — Re: Deploying the prototype as our production system

1. Non-functional requirements were deliberately left out. To build the prototype fast and cheaply, we intentionally skipped or relaxed error handling, input validation, security, performance under real load, and reliability under failure — none of these were prototyping objectives. A production system handling real data and real operations needs all of them properly engineered, not retrofitted.

2. Design quality was sacrificed for speed. Prototype code is written quickly, with shortcuts and little attention to structure, documentation, or maintainability. Continuing to pile features onto this foundation will make the system progressively harder, slower, and riskier to change — classic **technical debt**.

EXERCISE 2.8 — SOLVED (2/2)

Short Report — Continued, with Recommendation

3. The prototype's testers aren't representative, and it hasn't been properly tested. Prototype evaluators may not use the system the way real end-users will under production conditions, so lessons learned may not generalize. Critically, the prototype has **not** been through staged component/system/customer testing — only informal trial.

4. Requirements may still be wrong or incomplete. Part of a prototype's purpose is to **help discover** requirements — which means, by definition, some are probably not yet captured correctly. Building a full production system on an unclear requirements base risks very costly rework later.

5. There is little to no documentation for future maintainers, support staff, or new team members — a production system needs design, user, and maintenance documentation that a prototype never produces.

Recommendation: Use the prototype's validated concept and the lessons we've learned as input to a properly engineered process — for example, treating it as the **first increment** of an incremental-development process, re-engineered with proper design, documentation and testing. This costs more up front than shipping the prototype directly, but will save far more over the system's lifetime in reliability, maintainability, and reduced risk.